Documentație Proiect “Motoare de Grafică 3D în timp real”

Raport de Progres - Etapa 1

Realizat de: Codarcea Alexandru-Christian

Data: 09.11.2026

1. Introducere și Obiectivele Etapei 1

Scopul acestui proiect este dezvoltarea unui motor grafic 3D (3D Engine) modular, scalabil și performant, construit de la zero folosind limbajul C++. Etapa 1 a reprezentat fundația proiectului, în care s-au stabilit arhitectura de bază, pipeline-ul de randare, logica camerei și sistemul de importare a diverselor formate de modele 3D.

Spre deosebire de o abordare simplistică în care tot codul este scris într-un singur fișier (main.cpp) și greu de menținut, s-a optat pentru o abordare orientată pe obiecte. Acest document detaliază deciziile tehnice luate pentru a atinge cerințele impuse, combinând structura teoretică cu implementarea practică.

1.1. API 3D și Modul de Matematică (3D Math)

Pentru a garanta un control absolut asupra pipeline-ului grafic și a modului în care datele sunt trimise către placa video (GPU), proiectul utilizează următoarele tehnologii de bază:

- **API 3D (OpenGL):** S-a optat pentru OpenGL (Core Profile 3.3) pentru comunicarea directă cu placa video. Această versiune modernă impune folosirea shaderelor programabile (GLSL) și interzice funcțiile legacy, forțând dezvoltarea unui pipeline grafic eficient bazat pe Vertex Buffer Objects (VBO), Vertex Array Objects (VAO) și Element Buffer Objects (EBO).
- **Modul de 3D Math (GLM):** A fost integrată biblioteca GLM (OpenGL Mathematics), un standard în industrie. Aceasta gestionează toate operațiile matematice complexe necesare în grafica 3D: matrici de transformare (Model, View, Projection), calcule cu vectori (vec3, vec4), produse scalare/vectoriale și operații cu unghiuri (Euler angles).

2. Format Intern de Mesh-uri și Testarea Inițială

Unul dintre cele mai importante aspecte ale oricărui motor grafic este modul în care stochează și reprezintă geometria 3D în memoria RAM înainte de a o trimite către GPU. Motorul nostru definește un format intern propriu de abstractizare, structurat în clasele Mesh.h și Model.h.

2.1. Structura și Atributele Geometriei

Fiecare obiect din scenă este descompus într-un set de date standardizate, pentru a permite procesarea universală a oricărui tip de formă 3D:

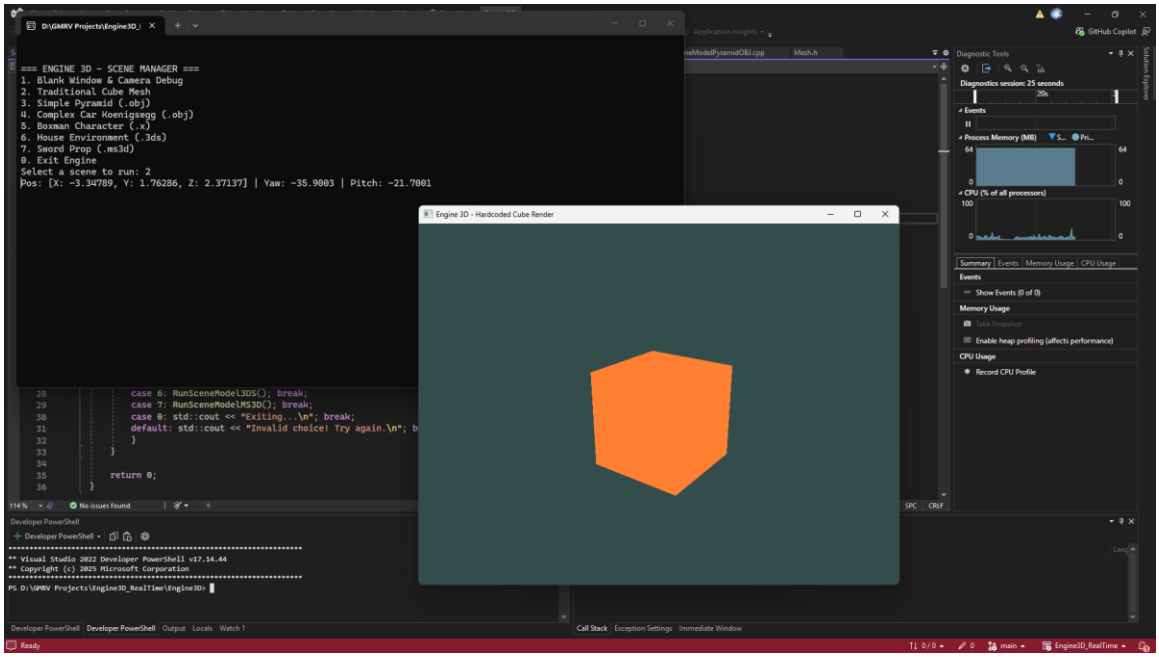
- **Triunghiuri (Indices):** Geometria este stocată și indexată prin vectori de indici. În loc să duplicăm vârfurile care se suprapun, folosim un Element Buffer Object (EBO). Această abordare conectează vârfurile sub formă de triunghiuri, optimizând drastic utilizarea memoriei în timpul apelului glDrawElements.
- **Diverse Atribute (Poziții, Normale, TexCoords):** Fiecare vârf este o structură complexă ce conține: coordonatele sale 3D (x, y, z), vectorii normali și coordonatele de textură (U, V) necesare pentru maparea precisă a imaginilor 2D pe suprafața 3D.
- **Texturi și Materiale:** Formatul suportă un vector de structuri de tip Texture, pregătind terenul pentru următoarea etapă. Fiecare material reține o cale către imagine pe disc și un ID generat de OpenGL, fiind capabil să diferențieze între diffuse maps, specular maps și normal maps.

2.2. Cubul Hardcodat și Universalitatea Shaderelor

Pentru a valida pipeline-ul OpenGL (crearea ferestrei, testul de adâncime Z-buffer, compilarea shaderelor), primul test a constat în randarea unui obiect primitiv: un cub ale cărui vârfuri și indici au fost introduși manual (hardcodat) în sursă.

Odată confirmată funcționalitatea cu acest cub, sistemul de shadere a fost rafinat. Numele fișierelor au fost schimbate din `CubeRender` în `basic.vert` și `basic.frag`.

Această redenumire reflectă tranziția către o abordare universală, în care un singur set de shadere de bază poate randa orice mesh, de la o piramidă simplă la o mașină complexă.

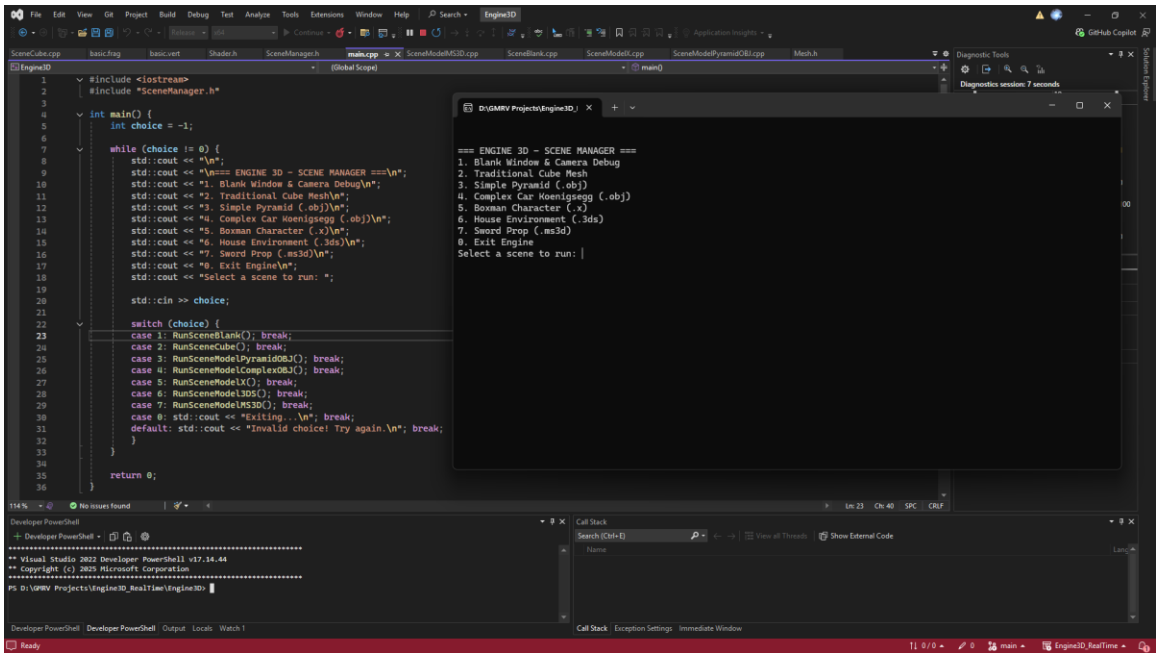


Figură 1: Randarea cubului hardcodat

3. Arhitectura Modulară: Scene Manager

Cea mai mare provocare în dezvoltarea unui motor 3D este gestionarea complexității. Dacă toată logica de desenare, definirea vârfurilor, configurarea camerei și încărcarea modelelor externe s-ar afla în `main.cpp`, codul ar deveni rapid imposibil de citit și extins.

Pentru a rezolva această problemă, a fost implementat un sistem modular numit Scene Manager. Fișierul `main.cpp` a fost transformat exclusiv într-un meniu CLI (Command Line Interface). Acest meniu întâmpină utilizatorul în consolă și îi permite să aleagă, la rulare, dintr-o listă de 7 scene diferite (Blank Screen, Cube, Modele Assimp).



Figură 2: Meniul interactiv cu selecția celor 7 opțiuni.

3.1. Izolarea Memoriei și Cuvântul Cheie 'static'

Fiecare scenă a fost decuplată într-un fișier separat (ex: SceneCube.cpp, SceneModelX.cpp). Această separare a generat inițial erori de tip "multiple definition" la nivel de linker, deoarece mai multe fișiere foloseau același nume pentru instanța camerei sau pentru funcțiile de input.

Soluția arhitecturală a fost utilizarea keyword-ului `static` în fața variabilelor globale și a funcțiilor ajutătoare din fiecare fișier de scenă. Aceasta limitează vizibilitatea variabilelor strict la fișierul curent (internal linkage). Astfel, fiecare scenă rulează în propriul său "sandbox", gestionându-și camerele și variabilele de stare independent de restul motorului.

4. Importer pentru Obiecte 3D

Punctul culminant al primei etape a fost integrarea Open Asset Import Library (Assimp) pentru a renunța la formele hardcodate și a permite ingestia de artă 3D reală. Motorul demonstrează o mare flexibilitate, fiind configurat și testat pentru a citi și procesa următoarele formate cunoscute:

• Format .obj (Wavefront) - Folosit pentru modelul complex Koenigsegg Agera și o piramidă simplă.

• Format .x (DirectX) - Folosit pentru modelul unui caracter (Boxman).

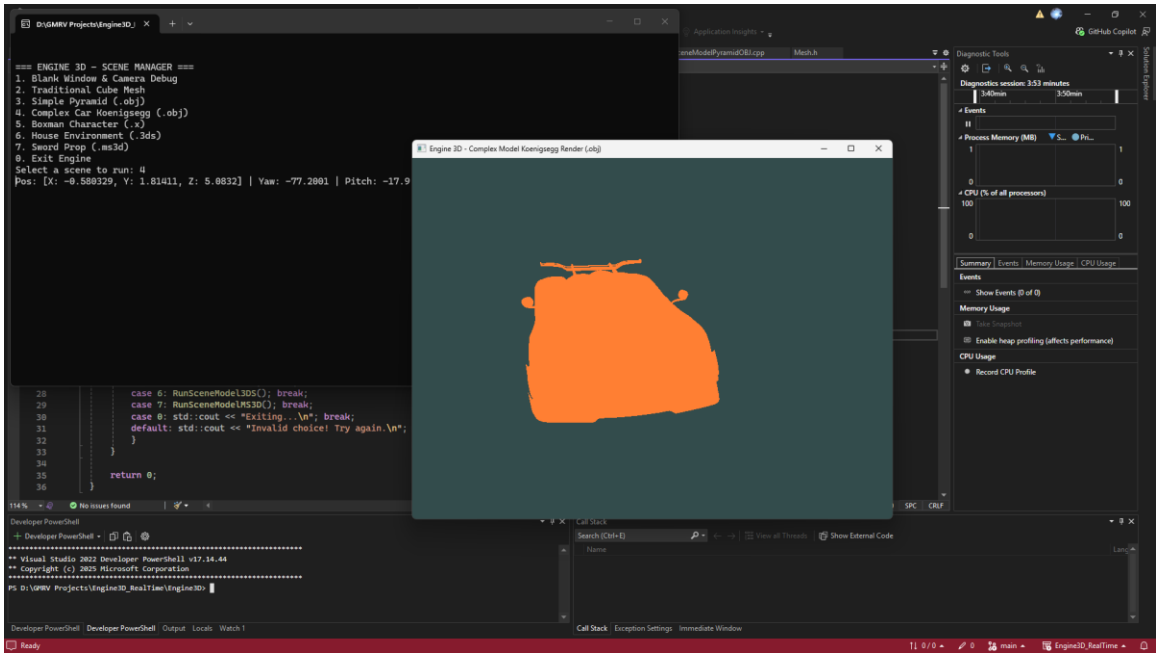
• Format .3ds (Autodesk 3D Studio) - Folosit pentru un environment detaliat (House).

• Format .ms3d (MilkShape 3D) - Folosit pentru modelul tip prop (Sword).

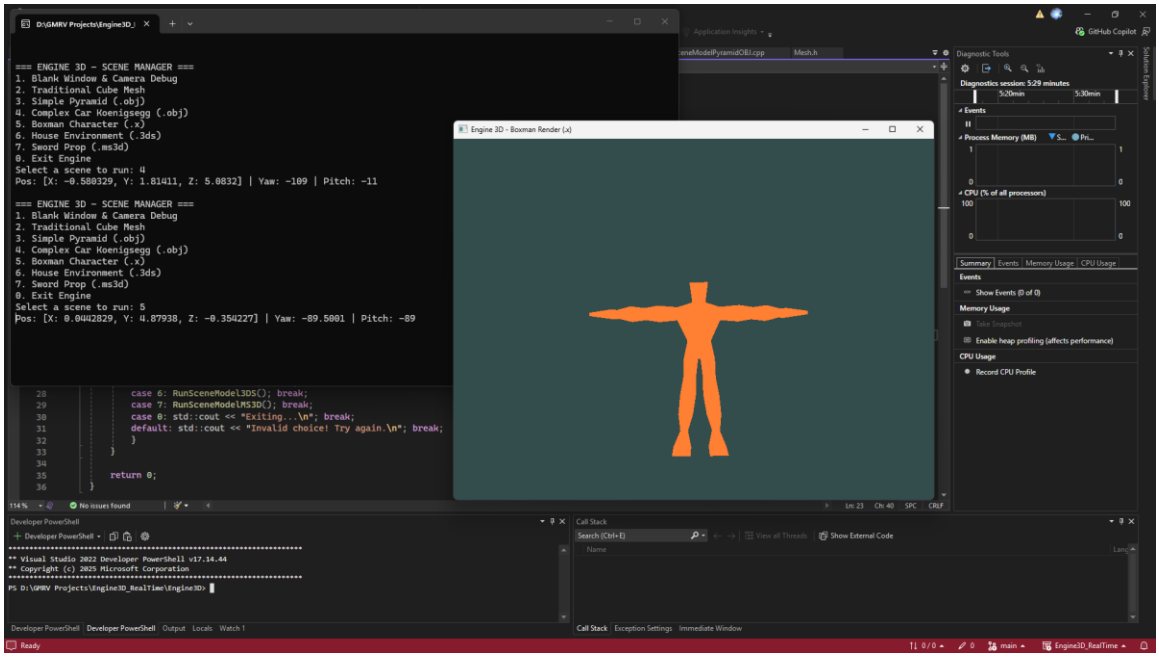
4.1. Corectarea Problemei de Scalare (Backface Culling)

Un detaliu tehnic interesant întâmpinat în faza de testare a modelelor descărcate de pe internet a fost faptul că acestea păreau a nu se randa pe ecran, deși datele erau citite corect de Assimp. Problema provenea din scara uriașă la care au fost modelate aceste obiecte (mii de unități OpenGL).

Având camera plasată la Z = 3.0, vizualizarea se afla blocată efectiv în interiorul modelului. Datorită optimizării de Backface Culling (care interzice desenarea fețelor interioare ale poligoanelor), modelul părea complet invizibil. Soluția aplicată a constat în aplicarea unei funcții de transformare `glm::scale` pe matricea de Model, reducând dimensiunile obiectelor, aducându-le perfect în câmpul vizual.



Figură 3: Modelul mașinii Koenigsegg afișat corect pe ecran.



Figură 4: Modelul Boxman.

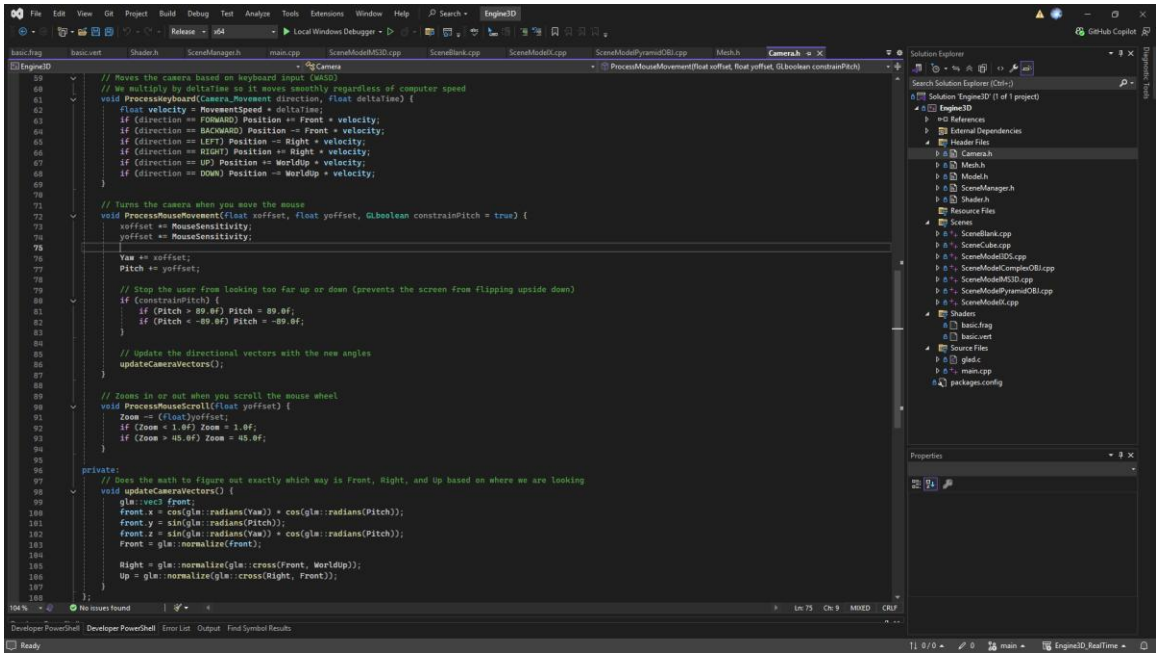
5. Interacțiunea cu Utilizatorul și Camera FPS

Sistemul de vizualizare folosește o implementare de tip First-Person Shooter (FPS), complet abstractizată în clasa Camera.h. Pentru a garanta o experiență fluidă indiferent de performanțele hardware ale computerului, toate calculele de mișcare sunt înmulțite cu `deltaTime` (timpul scurs între cadre).

5.1. Input/Output Events, Mouse și Keyboard

Gestiunea interacțiunii se face prin intermediul callback-urilor oferite de biblioteca GLFW:

- **Keyboard:** Apăsarea tastelor este interogată la fiecare cadru. S-au mapat tastele direcționale standard: W (Forward), S (Backward), A (Left), D (Right), completate de Q (Down) și E (Up) pentru libertate totală de mișcare pe cele 3 axe.
- **Mouse:** Mișcarea cursorului pe ecran este transformată instantaneu în unghiuri Euler (Pitch și Yaw), recalculând vectorii camerei (Front, Right, Up) folosind produse vectoriale (Cross Product). Rotița mouse-ului ajustează câmpul vizual (FOV), producând un efect natural de Zoom.



```
// Moves the camera based on keyboard input (WASD)
// We multiply by deltaTime so it moves smoothly regardless of computer speed
void ProcessKeyboard(Camera_Movement direction, float deltaTime) {
    float velocity = MovementSpeed * deltaTime;
    if (direction == FORWARD) Position += Front * velocity;
    if (direction == BACKWARD) Position -= Front * velocity;
    if (direction == LEFT) Position -= Right * velocity;
    if (direction == RIGHT) Position += Right * velocity;
    if (direction == UP) Position += WorldUp * velocity;
    if (direction == DOWN) Position -= WorldUp * velocity;
}

// Turns the camera when you move the mouse
void ProcessMouseMovement(float xoffset, float yoffset, GLboolean constrainPitch = true) {
    xoffset *= MouseSensitivity;
    yoffset *= MouseSensitivity;

    Yaw += xoffset;
    Pitch += yoffset;

    // Stop the user from looking too far up or down (prevents the screen from flipping upside down)
    if (constrainPitch) {
        if (Pitch > 89.0f) Pitch = 89.0f;
        if (Pitch < -89.0f) Pitch = -89.0f;
    }

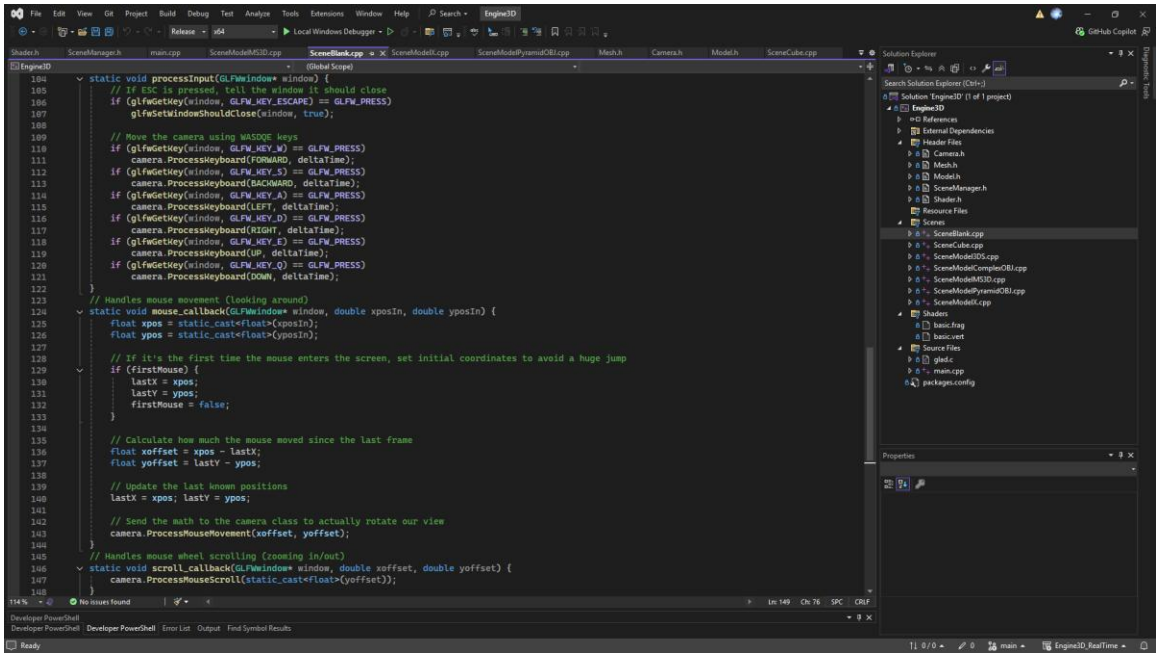
    // Update the directional vectors with the new angles
    updateCameraVectors();
}

// Zooms in or out when you scroll the mouse wheel
void ProcessMouseScroll(float yoffset) {
    Zoom -= (float)yoffset;
    if (Zoom < 1.0f) Zoom = 1.0f;
    if (Zoom > 45.0f) Zoom = 45.0f;
}

private:
// Does the math to figure out exactly which way is Front, Right, and Up based on where we are looking
void updateCameraVectors() {
    glm::vec3 front;
    front.x = cos(glm::radians(Yaw)) * cos(glm::radians(Pitch));
    front.y = sin(glm::radians(Pitch));
    front.z = sin(glm::radians(Yaw)) * cos(glm::radians(Pitch));
    Front = glm::normalize(front);

    Right = glm::normalize(glm::cross(Front, WorldUp));
    Up = glm::normalize(glm::cross(Right, Front));
}
};
```

Figură 5: Camera.h.



```
static void processInput(GLFWwindow* window) {
    // If ESC is pressed, tell the window it should close
    if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
        glfwSetWindowShouldClose(window, true);

    // Move the camera using WASDQE keys
    if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS)
        camera.ProcessKeyboard(FORWARD, deltaTime);
    if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS)
        camera.ProcessKeyboard(BACKWARD, deltaTime);
    if (glfwGetKey(window, GLFW_KEY_A) == GLFW_PRESS)
        camera.ProcessKeyboard(LEFT, deltaTime);
    if (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS)
        camera.ProcessKeyboard(RIGHT, deltaTime);
    if (glfwGetKey(window, GLFW_KEY_E) == GLFW_PRESS)
        camera.ProcessKeyboard(UP, deltaTime);
    if (glfwGetKey(window, GLFW_KEY_Q) == GLFW_PRESS)
        camera.ProcessKeyboard(DOWN, deltaTime);
}
// Handles mouse movement (looking around)
static void mouse_callback(GLFWwindow* window, double xposIn, double yposIn) {
    float xpos = static_cast<float>(xposIn);
    float ypos = static_cast<float>(yposIn);

    // If it's the first time the mouse enters the screen, set initial coordinates to avoid a huge jump
    if (firstMouse) {
        lastX = xpos;
        lastY = ypos;
        firstMouse = false;
    }

    // Calculate how much the mouse moved since the last frame
    float xoffset = xpos - lastX;
    float yoffset = lastY - ypos;

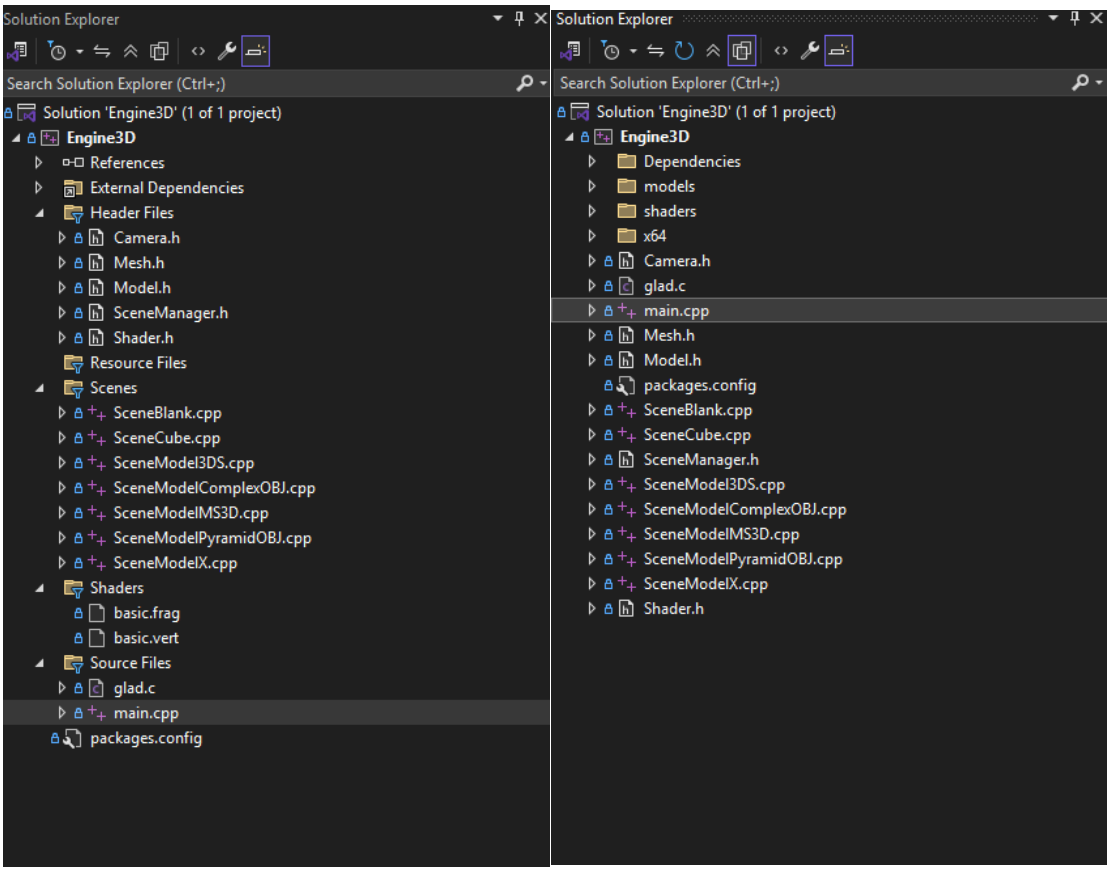
    // Update the last known positions
    lastX = xpos; lastY = ypos;

    // Send the math to the camera class to actually rotate our view
    camera.ProcessMouseMovement(xoffset, yoffset);
}
// Handles mouse wheel scrolling (zooming in/out)
static void scroll_callback(GLFWwindow* window, double xoffset, double yoffset) {
    camera.ProcessMouseScroll(static_cast<float>(yoffset));
}
```

Figură 6: Procesarea evenimentelor de Mouse & Keyboard.

6. Organizarea Proiectului (Visual Studio, Git și SVN)

Un ultim aspect crucial din această primă etapă este organizarea soluției de cod. S-a decis păstrarea tuturor fișierelor C++ (sursă și headere) într-o structură plată pe hard disk, evitând complexitatea adresărilor relative (de tip `../`). Totuși, în mediul Visual Studio, proiectul este impecabil organizat prin Filtre Virtuale, separând codul sursă de resursele statice (models, shaders).



Figură 7: Organizarea fișierelor.

Pentru gestionarea versiunilor s-a adoptat o abordare hibridă, utilizând în paralel atât Git (pentru urmărirea granulară a pașilor de dezvoltare pe GitHub), cât și SVN (pentru

livrarea oficială a etapelor proiectului). Pentru ca cele două sisteme să coexiste în același director fără conflicte, au fost configurate fișierele `.gitignore` și `.svnignore` (aplicat prin proprietățile `svn:ignore` și `svn:global-ignores`), astfel încât Git să ignore metadatele `.svn/`, iar SVN să ignore metadatele `.git/`, alături de directoarele de cache și compilare generate de Visual Studio (`.vs/`, `x64/`, `Debug/`, `Release/`, `packages/`). De asemenea, în `.gitignore` a fost introdusă regula `!**/models/*.obj` pentru a preveni excluderea modelelor 3D Wavefront (pe care sistemele de versionare le pot confunda în mod prestabilit cu fișierele binare de tip Object rezultate din compilare).